Diabetes Diagnosis with Machine Learning

Author: Miguel Castro — Machine Learning, PEC4

This project compares six supervised classifiers for predicting diabetes from clinical measurements, evaluating each on accuracy, precision, and recall.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

1. Data

```
# read the tab-separated file
df = pd.read_csv('patients_diabetis.txt', sep='\t')
df.head()
```

The dataset has 768 patients, eight clinical predictors (pregnancies, glucose, blood pressure, triceps skinfold, insulin, BMI, diabetes pedigree, age) and a binary `class` label. The class distribution is imbalanced: 500 negatives and 268 positives.

```
# statistical summary
print(df.describe())
# class distribution
print(df['class'].value_counts())
# correlation heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation map")
plt.show()
```

[Figure. Correlation heatmap of the predictors.]

2. Preprocessing

```
# predictors and label
X = df.drop('class', axis=1)
y = df['class']

# standardisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 67% train / 33% test split, seed 12345, stratified
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.33, random_state=12345, stratify=y
)
```

Features are standardised, and the split is stratified so both classes keep their proportions in train and test.

```
        # helper: fit, predict, and return accuracy, precision and recall
        def metrics(model):
            model.fit(X_train, y_train)
            y_pred = model.predict(X_test)
            return {
                'Accuracy':  accuracy_score(y_test, y_pred),
                'Precision': precision_score(y_test, y_pred),
                'Recall':    recall_score(y_test, y_pred)
            }
```

3. Models

3.1 k-NN

```
        # k-nn with k = 1, 3, 5, 7, 11
        results_knn = []
        for k in [1, 3, 5, 7, 11]:
            knn = KNeighborsClassifier(n_neighbors=k)
            res = metrics(knn); res['k'] = k
            results_knn.append(res)
```

```
   k  Accuracy  Precision    Recall
   1  0.720472   0.604651  0.584270
   3  0.740157   0.653333  0.550562
   5  0.732283   0.647887  0.516854
   7  0.748031   0.671233  0.550562
  11  0.728346   0.661290  0.460674
```

Comment: k = 5 offers the best balance between precision and recall.

3.2 Naive Bayes

```
        res = metrics(GaussianNB())
```

```
   Accuracy  Precision   Recall
   0.748031   0.658228  0.58427
```

Comment: Naive Bayes is fast, but with lower recall than other models.

3.3 Neural network (MLP)

```
        # neural network with three architectures
        configs = [(12,6), (20,10), (20,10,5)]
        results_mlp = []
        for layers in configs:
            mlp = MLPClassifier(hidden_layer_sizes=layers, max_iter=1000, random_state=123)
            res = metrics(mlp); res['Architecture'] = str(layers)
            results_mlp.append(res)
```

```
  Architecture  Accuracy  Precision    Recall
      (12, 6)   0.748031   0.647059  0.617978
     (20, 10)   0.708661   0.577320  0.629213
   (20, 10, 5)  0.677165   0.538462  0.550562
```

Comment: the (20,10,5) architecture has better recall, though it trains more slowly.

3.4 SVM

```
        # svm with linear and rbf kernels
        kernels = ['linear', 'rbf']
        results_svm = []
        for k in kernels:
```

```
    svm = SVC(kernel=k, random_state=123)
    res = metrics(svm); res['Kernel'] = k
    results_svm.append(res)
```

```
Kernel  Accuracy  Precision    Recall
linear   0.76378   0.704225  0.561798
   rbf   0.76378   0.683544  0.606742
```

Comment: the RBF kernel slightly improves recall over the linear one.

3.5 Decision tree

```
res = metrics(DecisionTreeClassifier(random_state=123))
```

```
Accuracy  Precision   Recall
0.645669   0.495238  0.58427
```

Comment: a simple tree, easy to interpret, but with weaker generalisation.

3.6 Random forest

```
# random forest with 100 and 200 trees
results_rf = []
for n in [100, 200]:
    rf = RandomForestClassifier(n_estimators=n, random_state=123)
    res = metrics(rf); res['Trees'] = n
    results_rf.append(res)
```

```
Trees  Accuracy  Precision    Recall
 100   0.73622   0.644737  0.550562
 200   0.73622   0.648649  0.539326
```

Comment: RF with 200 trees slightly improves recall without overfitting.

4. Summary and conclusion

```
# summary table sorted by accuracy
summary = (pd.DataFrame(results)[['Model', 'Accuracy', 'Precision', 'Recall']]
             .sort_values('Accuracy', ascending=False)
             .reset_index(drop=True))
print(summary)
```

Representative result per algorithm (best-accuracy variant of each family), from the reproducible model outputs above:

Algorithm	Accuracy	Precision	Recall
SVM (RBF)	0.764	0.684	0.607
k-NN (k=7)	0.748	0.671	0.551
Naive Bayes	0.748	0.658	0.584
Neural network (12,6)	0.748	0.647	0.618
Random forest (200)	0.736	0.649	0.539
Decision tree	0.646	0.495	0.584

Conclusion: no single classifier dominates across all three metrics. The RBF-kernel SVM gives the best test accuracy (0.76) and the strongest recall among the top-accuracy models (0.61). k-NN, naive Bayes, and a small neural network cluster just behind at ~0.75 accuracy, and random forest sits mid-pack (0.74 accuracy, 0.54 recall). The single highest recall comes from a larger neural network (~0.63), but at a clear cost to accuracy.

Recall is the metric to weigh most heavily here: in a screening context a false negative — a missed case — is the costly error, so the accuracy/recall trade-off matters more than any single headline figure. On that basis the RBF SVM is the most balanced choice, while a recall-first deployment might prefer a neural network and accept lower overall accuracy.